

PRACTICAL-7

AIM:

Wap to create a string like, "Hello World" and perform some operations.

- (A) To perform all indexing / slicing operations.
- (B) To perform update operation in string.
- (C) To perform upper case and lower case operations.
- (D) To perform delete operations.

CODE:

```
#Creating the string
text = "Hello World"
# Indexing / Slicing operations
print("Indexing Operations:")
print("First character:", text[0])
print("Last character:", text[-1])
print("Characters from index 0 to 4:", text[0:5])
print("Characters from index 6 to end:", text[6:])
#Update operation in string
updatedtext = text[:6] + "Universe"
print("\nUpdated Text:", updatedtext)
#Upper case and lower case operations
print("Upper case:", text.upper())
print("Lower case:", text.lower())
# Delete operations
text_after_delete = text.replace("World", "")
print("\nAfter deleting 'World':", text_after_delete)
```

OUTPUT:

```
Indexing Operations:
First character: H
Last character: d
Characters from index 0 to 4: Hello
Characters from index 6 to end: World

Updated Text: Hello Universe

Upper case: HELLO WORLD
Lower case: hello world

After deleting 'World': Hello
```

PRACTICAL-8

AIM:

Wap to create the list like [1,2,3, "Rohit","Ankit",True] and perform some operations.

- (A) To perform all indexing/slicing operations in above list.
- (B) To perform all update operations in above list.
- (C) To perform all Delete operation in above list.
- (D) To perform all reverse operations in above list.

CODE:

```
# Creating the list
my_list = [1, 2, 3, "Rohit", "Ankit", True]
# (A) Indexing / Slicing operations
print("Indexing Operations:")
print("First element:", my_list[0])
print("Last element:", my_list[-1])
print("Elements from index 1 to 3:", my_list[1:4])
print("Elements from index 2 to the end:", my_list[2:])
# (B) Update operations in the list
print("\nUpdate Operations:")
my_list[1] = 10
my_list[5] = False
print("Updated List:", my_list)
# (C) Delete operations in the list
print("\nDelete Operations:")
del my_list[2]
print("List after deletion at index 2:", my_list)
my_list.remove("Rohit")
print("List after removing 'Rohit':", my_list)
my_list.pop()
print("List after popping the last element:", my_list)
# (D) Reverse operations on the list
print("\nReverse Operations:")
my_list.reverse()
print("Reversed List:", my_list)
```

OUTPUT:

Indexing Operations:

First element: 1

Last element: True

Elements from index 1 to 3: [2, 3, 'Rohit']

Elements from index 2 to the end: [3, 'Rohit', 'Ankit', True]

Update Operations:

Updated List: [1, 10, 3, 'Rohit', 'Ankit', False]

Delete Operations:

List after deletion at index 2: [1, 10, 'Rohit', 'Ankit', False]

List after removing 'Rohit': [1, 10, 'Ankit', False]

List after popping the last element: [1, 10, 'Ankit']

Reverse Operations:

Reversed List: ['Ankit', 10, 1]

PRACTICAL-9

AIM:

Wap to Perform the Sum, Len, Avg, Min, Max Method in the list.

CODE:

```
# Creating the list
my_list = [1, 2, 3, 4, 5]
# (A) Sum of the list
sum_of_list = sum(my_list)
print("Sum of the list:", sum_of_list)
# (B) Length of the list
len_of_list = len(my_list)
print("Length of the list:", len_of_list)
# (C) Average of the list
avg_of_list = sum_of_list / len_of_list
print("Average of the list:", avg_of_list)
# (D) Minimum value in the list
min_of_list = min(my_list)
print("Minimum value in the list:", min_of_list)
# (E) Maximum value in the list
max_of_list = max(my_list)
print("Maximum value in the list:", max_of_list)
```

OUTPUT:

```
Sum of the list: 15
Length of the list: 5
Average of the list: 3.0
Minimum value in the list: 1
Maximum value in the list: 5
```

PRACTICAL-10

AIM:

Wap to perform all possible Operations in Tuple.

CODE:

```
tuple1 = (1, 2, 3, 4, 5)
tuple2 = ('a', 'b', 'c')

#Access elements by index
print("Accessing elements by index:")
print("First element of tuple1:", tuple1[0])
print("Last element of tuple2:", tuple2[-1])

#Slice a tuple
print("\nSlicing a tuple:")
print("Slice from index 1 to 3 of tuple1:", tuple1[1:4])
print("Slice of all elements in tuple2:", tuple2[:])

#Concatenate two tuples
print("\nConcatenating tuples:")
tuple3 = tuple1 + tuple2
print("Concatenated tuple:", tuple3)

#Repeat a tuple
print("\nRepeating a tuple:")
tuple4 = tuple1 * 2
print("Repeated tuple1:", tuple4)

#Get length of a tuple
print("\nLength of tuple1:", len(tuple1))
print("Length of tuple2:", len(tuple2))
```

OUTPUT:

```
Accessing elements by index:  
First element of tuple1: 1  
Last element of tuple2: c  
  
Slicing a tuple:  
Slice from index 1 to 3 of tuple1: (2, 3, 4)  
Slice of all elements in tuple2: ('a', 'b', 'c')  
  
Concatenating tuples:  
Concatenated tuple: (1, 2, 3, 4, 5, 'a', 'b', 'c')  
  
Repeating a tuple:  
Repeated tuple1: (1, 2, 3, 4, 5, 1, 2, 3, 4, 5)  
  
Length of tuple1: 5  
Length of tuple2: 3
```

PRACTICAL-11

AIM:

Wap to Perform all possible Operations in Dictionary.

CODE:

```
dict1 = {'name': 'John', 'age': 25, 'city': 'New York'}
dict2 = {'course': 'Python', 'duration': '3 months'}
# Access values using keys
print("Name:", dict1['name'])
# Add a new key-value pair
dict1['email'] = 'john@example.com'
print("Updated dict1:", dict1)
# Modify an existing key's value
dict1['age'] = 26
print("Updated dict1 with modified age:", dict1)
# using `del`
del dict1['city']
print("dict1 after removing 'city':", dict1)
# using `pop()`
removed_value = dict2.pop('duration')
print("Removed 'duration':", removed_value)
print("Updated dict2:", dict2)
# Get the value of a key using `get()`
print("Get value of 'age':", dict1.get('age'))
print("Get value of 'email':", dict1.get('email'))
# Get all keys or values using `keys()` and `values()`
print("Keys of dict1:", dict1.keys())
print("Values of dict1:", dict1.values())
# length of a dictionary
print("Length of dict1:", len(dict1))
# Clear all items from the dictionary
dict1.clear()
print("dict1 after clearing all items:", dict1)
```

OUTPUT:

```
dict1 = {'name': 'John', 'age': 25, 'city': 'New York'}
dict2 = {'course': 'Python', 'duration': '3 months'}

# Access values using keys
print("Name:", dict1['name'])

# Add a new key-value pair
dict1['email'] = 'john@example.com'
print("Updated dict1:", dict1)

# Modify an existing key's value
dict1['age'] = 26
print("Updated dict1 with modified age:", dict1)

# using `del`
del dict1['city']
print("dict1 after removing 'city':", dict1)

# using `pop()`
removed_value = dict2.pop('duration')
print("Removed 'duration':", removed_value)
print("Updated dict2:", dict2)

# Get the value of a key using `get()`
print("Get value of 'age':", dict1.get('age'))
print("Get value of 'email':", dict1.get('email'))

# Get all keys or values using `keys()` and `values()`
print("Keys of dict1:", dict1.keys())
print("Values of dict1:", dict1.values())

# length of a dictionary
print("Length of dict1:", len(dict1))

# Clear all items from the dictionary
dict1.clear()
print("dict1 after clearing all items:", dict1)
```


PRACTICAL-12

AIM:

Wap to Perform all possible Operations in set .

CODE:

```
# Create a set
set1 = {1, 2, 3, 4, 5}
set2 = {4, 5, 6, 7, 8}
print("Set 1:", set1)
print("Set 2:", set2)

# Add element
set1.add(10)
print("\nAfter adding 10 to Set1:", set1)

# Remove element
set1.remove(2)
print("After removing 2 from Set1:", set1)

# Clear set
set3 = {9, 10}
set3.clear()
print("After clearing set3:", set3)
```

OUTPUT:

```
Set 1: {1, 2, 3, 4, 5}
Set 2: {4, 5, 6, 7, 8}

After adding 10 to Set1: {1, 2, 3, 4, 5, 10}
After removing 2 from Set1: {1, 3, 4, 5, 10}
After clearing set3: set()
```

PRACTICAL-13

AIM:

WAP to create a basic calculator with the help of functions.

CODE:

```
def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def multiply(x, y):
    return x * y

def divide(x, y):
    if y == 0:
        return "Cannot divide by zero!"
    return x / y

print("Select operation:")
print("1. Add")
print("2. Subtract")
print("3. Multiply")
print("4. Divide")

choice = input("Enter choice (1/2/3/4): ")

num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

if choice == '1':
    print("Result:", add(num1, num2))
elif choice == '2':
    print("Result:", subtract(num1, num2))
elif choice == '3':
    print("Result:", multiply(num1, num2))
elif choice == '4':
```

```
    print("Result:", divide(num1, num2))  
else:  
    print("Invalid input")
```

OUTPUT:

```
Select operation:  
1. Add  
2. Subtract  
3. Multiply  
4. Divide  
Enter choice (1/2/3/4): 3  
Enter first number: 5  
Enter second number: 6  
Result: 30.0
```

PRACTICAL-14

AIM:

WAP to implement basic calculator with the help of class & Object.

CODE:

```
class Calculator:
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def add(self):
        return self.a + self.b

    def subtract(self):
        return self.a - self.b

    def multiply(self):
        return self.a * self.b

    def divide(self):
        if self.b == 0:
            return "Cannot divide by zero!"
        return self.a / self.b

# --- Main Program ---

print("Basic Calculator")
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

# Create object
calc = Calculator(a, b)

print("\nChoose Operation:")
print("1. Add")
print("2. Subtract")
print("3. Multiply")
```

```
print("4. Divide")
```

```
choice = input("Enter choice (1/2/3/4): ")
```

```
if choice == '1':
```

```
    print("Result:", calc.add())
```

```
elif choice == '2':
```

```
    print("Result:", calc.subtract())
```

```
elif choice == '3':
```

```
    print("Result:", calc.multiply())
```

```
elif choice == '4':
```

```
    print("Result:", calc.divide())
```

```
else:
```

```
    print("Invalid choice")
```

OUTPUT:

```
Basic Calculator
Enter first number: 10
Enter second number: 5

Choose Operation:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice (1/2/3/4): 1
Result: 15.0
```

PRACTICAL-15

AIM:

WAP to implement Inheritance With it's all types.

Example:

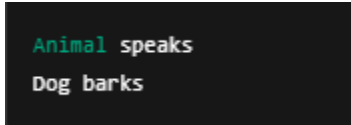
- > Single inheritance.
- > Multiple inheritance.
- > Multilevel inheritance.
- > hybrid inheritance.
- > hierarchical inheritance.

CODE:

Single Inheritance: One parent, one child

```
class Animal:
    def speak(self):
        print("Animal speaks")
class Dog(Animal):
    def bark(self):
        print("Dog barks")
d = Dog()
d.speak()
d.bark()
```

OUTPUT:



```
Animal speaks
Dog barks
```

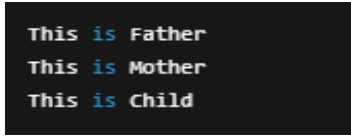
CODE:

Multiple Inheritance: One child class inherits from two parent classes

```
class Father:
    def showfather(self):
        print("This is Father")
class Mother:
    def showmother(self):
        print("This is Mother")
```

```
class Child(Father, Mother):
    def showchild(self):
        print("This is Child")
c = Child()
c.showfather()
c.showmother()
c.showchild()
```

OUTPUT:



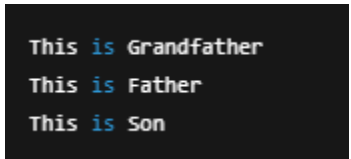
```
This is Father
This is Mother
This is Child
```

CODE:

Multilevel Inheritance: Grandparent → Parent → Child

```
class Grandfather:
    def showgrandfather(self):
        print("This is Grandfather")
class Father(Grandfather):
    def showfather(self):
        print("This is Father")
class Son(Father):
    def showson(self):
        print("This is Son")
s = Son()
s.showgrandfather()
s.showfather()
s.showson()
```

OUTPUT:



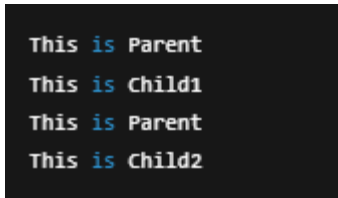
```
This is Grandfather
This is Father
This is Son
```

CODE:

Hierarchical Inheritance: One parent, multiple children

```
class Parent:
    def showparent(self):
        print("This is Parent")
class Child1(Parent):
    def showchild1(self):
        print("This is Child1")
class Child2(Parent):
    def showchild2(self):
        print("This is Child2")
c1 = Child1()
c2 = Child2()
c1.showparent()
c1.showchild1()
c2.showparent()
c2.showchild2()
```

OUTPUT:



```
This is Parent
This is Child1
This is Parent
This is Child2
```

CODE:

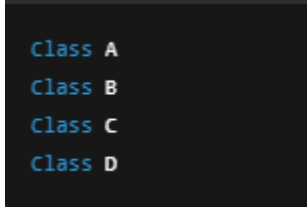
Hybrid Inheritance: Combination of more than one type (here: Multiple + Hierarchical)

```
class A:
    def showa(self):
        print("Class A")
class B(A):
    def showb(self):
        print("Class B")
class C(A):
    def showc(self):
        print("Class C")
class D(B, C):
```



```
def showd(self):  
    print("Class D")  
d = D()  
d.showa()  
d.showb()  
d.showc()  
d.showd()
```

OUTPUT:



```
Class A  
Class B  
Class C  
Class D
```

PRACTICAL-16

AIM:

WAP to implement polymorphism With it's all types.

CODE:

1. Duck Typing

```
class Dog:
```

```
    def sound(self):  
        print("Bark")
```

```
class Cat:
```

```
    def sound(self):  
        print("Meow")
```

```
def make_sound(animal):
```

```
    animal.sound()
```

2. Operator Overloading

```
class Point:
```

```
    def __init__(self, x):  
        self.x = x
```

```
    def __add__(self, other):  
        return Point(self.x + other.x)
```

```
    def __str__(self):  
        return f"{self.x}"
```

3. Method Overloading (simulated)

```
class Multiply:
```

```
    def product(self, a=None, b=None):  
        if a and b:  
            return a * b  
        elif a:  
            return a  
        else:
```

```
    return 0
```

4. Method Overriding

```
class Animal:
```

```
    def speak(self):
```

```
        print("Animal sound")
```

```
class DogChild(Animal):
```

```
    def speak(self):
```

```
        print("Woof!")
```

Running the examples

```
print("Duck Typing Example:")
```

```
make_sound(Dog())
```

```
make_sound(Cat())
```

```
print("\nOperator Overloading Example:")
```

```
p1 = Point(5)
```

```
p2 = Point(10)
```

```
print("Sum of points:", p1 + p2)
```

```
print("\nMethod Overloading (simulated):")
```

```
m = Multiply()
```

```
print(m.product(4, 5))
```

```
print(m.product(3))
```

```
print("\nMethod Overriding Example:")
```

```
a = Animal()
```

```
d = DogChild()
```

```
a.speak()
```

```
d.speak()
```

OUTPUT:

Duck Typing Example:

Bark

Meow

Operator Overloading Example:

Sum of points: 15

Method Overloading (simulated):

20

3

Method Overriding Example:

Animal sound

Woof!

PRACTICAL-17

AIM:

WAP to Implementing Abstraction With it's all types.

CODE:

```
from abc import ABC, abstractmethod
```

```
# 1. Abstract Class & Abstract Method
```

```
class Vehicle(ABC):
```

```
    @abstractmethod
```

```
    def start(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def stop(self):
```

```
        pass
```

```
# 2. Partial Abstraction (has both abstract and concrete methods)
```

```
class Car(Vehicle):
```

```
    def start(self):
```

```
        print("Car started")
```

```
    def stop(self):
```

```
        print("Car stopped")
```

```
    def music(self):
```

```
        print("Playing music in car")
```

```
# 3. Interface Simulation (All methods are abstract)
```

```
class RemoteControl(ABC):
```

```
    @abstractmethod
```

```
    def turn_on(self):
```

```
        pass
```

```
    @abstractmethod
```

```
def turn_off(self):  
    pass  
  
# Implementing the interface  
class TV(RemoteControl):  
    def turn_on(self):  
        print("TV is ON")  
  
    def turn_off(self):  
        print("TV is OFF")  
  
# Running the examples  
print("Using Abstract Class:")  
c = Car()  
c.start()  
c.music()  
c.stop()  
  
print("\nUsing Interface Simulation:")  
t = TV()  
t.turn_on()  
t.turn_off()
```

OUTPUT:

```
Using Abstract Class:  
Car started  
Playing music in car  
Car stopped  
  
Using Interface Simulation:  
TV is ON  
TV is OFF
```

PRACTICAL-18

AIM:

WAP to Implementing Encapsulation With it's all types.

CODE:

```
class Person:
    # Public member
    def __init__(self, name, age):
        self.name = name        # public
        self._salary = 50000    # protected
        self.__bank_balance = 100000 # private

    def show_info(self):
        print(f"Name: {self.name}")
        print(f"Salary (Protected): {self._salary}")
        print(f"Bank Balance (Private): {self.__bank_balance}")

    # Public method to access private data
    def get_balance(self):
        return self.__bank_balance

    # Public method to modify private data
    def set_balance(self, amount):
        if amount > 0:
            self.__bank_balance = amount
        else:
            print("Invalid balance amount!")

# Creating an object
p = Person("Alice", 30)

print("Accessing Public Member:")
print(p.name)
```

OUTPUT:

```
Accessing Public Member:
```

```
Alice
```

```
Accessing Protected Member:
```

```
50000
```

```
Trying to Access Private Member Directly:
```

```
Cannot access private variable directly!
```

```
Accessing Private Member via Public Method:
```

```
100000
```

```
Modifying Private Member via Setter:
```

```
150000
```